Java Solutions - First 5 Coding Problems

Problem 1: EcoSmart Home Hub Controller

```
abstract class SmartDevice {
    protected String deviceId, deviceName;
    SmartDevice(String id,String name){deviceId=id;deviceName=name;}
    public abstract void runDiagnostic();
}
interface BatteryOperated{
    int getBatteryLevel();
    void triggerRechargeAlert();
}
class SmartLight extends SmartDevice{
    SmartLight(String i,String n){super(i,n);}
    public void runDiagnostic(){System.out.println(deviceName+" Light Dia
gnostic");}
}
class SmartCamera extends SmartDevice implements BatteryOperated{
    int battery;
    SmartCamera(String i,String n,int b){super(i,n);battery=b;}
    public void runDiagnostic(){System.out.println(deviceName+" Camera Di
agnostic");}
    public int getBatteryLevel(){return battery;}
    public void triggerRechargeAlert(){System.out.println(deviceName+" Re
charge Alert!");}
}
class SmartLock extends SmartDevice implements BatteryOperated{
    int battery;
    SmartLock(String i,String n,int b){super(i,n);battery=b;}
    public void runDiagnostic(){System.out.println(deviceName+" Lock Diag
nostic");}
    public int getBatteryLevel(){return battery;}
    public void triggerRechargeAlert(){System.out.println(deviceName+" Re
charge Alert!");}
}
class HomeHub{
    public void executeNightlyRoutine(SmartDevice[] devices){
        for(SmartDevice d:devices){
            d.runDiagnostic();
            if(d instanceof BatteryOperated){
                BatteryOperated b=(BatteryOperated)d;
                if(b.getBatteryLevel()<20) b.triggerRechargeAlert();
            }
        }
    }
}
```

Problem 2: AeroLogix Drone Fleet

```
abstract class DeliveryDrone{
    String droneId;
    DeliveryDrone(String id){droneId=id;}
    abstract void deliverPackage();
}
interface Airborne{
```

```
    void flyToDestination();
    default void requestAirTrafficClearance(){
        System.out.println("Air Traffic Clearance Granted");
    }
}
interface GroundBased{void navigateSidewalks();}
class Quadcopter extends DeliveryDrone implements Airborne{
    Quadcopter(String id){super(id);}
    public void flyToDestination(){System.out.println("Flying");}
    public void deliverPackage(){requestAirTrafficClearance();flyToDestin
ation();}
}
class CityRover extends DeliveryDrone implements GroundBased{
    CityRover(String id){super(id);}
    public void navigateSidewalks(){System.out.println("Driving");}
    public void deliverPackage(){navigateSidewalks();}
}
class HybridVTOL extends DeliveryDrone implements Airborne,GroundBased{
    HybridVTOL(String id){super(id);}
    public void flyToDestination(){System.out.println("Flying");}
    public void navigateSidewalks(){System.out.println("Driving");}
    public void deliverPackage(){requestAirTrafficClearance();flyToDestin
ation();navigateSidewalks();}
}
```

Problem 3: VaultGuard State Machine

```
enum DoorState{OPEN,CLOSED,LOCKED}
class IllegalStateTransitionException extends RuntimeException{
    IllegalStateTransitionException(String msg){super(msg);}
}
class VaultDoor{
    private DoorState state=DoorState.OPEN;
    public void closeDoor(){state=DoorState.CLOSED;}
    public void lockDoor(){
        if(state!=DoorState.CLOSED)
            throw new IllegalStateTransitionException("Cannot lock an ope
n door");
        state=DoorState.LOCKED;
    }
    public void unlockDoor(){
        if(state==DoorState.LOCKED) state=DoorState.CLOSED;
    }
}
```

Problem 4: FinTech Routing Engine

```
interface PaymentStrategy{
    boolean processPayment(double amount);
}
class CreditCardStrategy implements PaymentStrategy{
    public boolean processPayment(double amount){
        System.out.println("Credit Card Payment: "+amount);
        return true;
    }
}
```

```
class CryptoStrategy implements PaymentStrategy{
    public boolean processPayment(double amount){
        System.out.println("Crypto Payment: "+amount);
        return true;
    }
}
class TransactionProcessor{
    private PaymentStrategy strategy;
    TransactionProcessor(PaymentStrategy s){strategy=s;}
    public void setPaymentStrategy(PaymentStrategy s){strategy=s;}
    public void executeTransaction(double amount){strategy.processPayment
(amount);}
}
```

Problem 5: DataStream ETL Pipeline

```
abstract class DataPayload{
    abstract String getRawContent();
}
class JsonPayload extends DataPayload{
    public String getRawContent(){return "{name:'Abhishek'}";}
}
class XmlPayload extends DataPayload{
    public String getRawContent(){return "<name>Abhishek</name>";}
}
class PipelineProcessor<T extends DataPayload>{
    public void process(T payload){
        System.out.println(payload.getRawContent());
    }
}
```